



# Funções em C: Estrutura, Escopo e Modularização

Organização do código estruturado.

---

# Introdução à Funções

---

Consiste em um conjunto de comandos agrupados para realizar uma tarefa.

Recebem um nome pelo qual podem ser invocada em diversas partes do programa.

**Retornam um valor**, o qual pode ser armazenado em uma variável do mesmo tipo no ponto de chamada.

A principal finalidade é a **modularização** de um programa, permitindo dividir um sistema em partes menores, onde cada função realiza uma tarefa bem definida.

---

# O Esqueleto de uma Função em C

## Sintaxe e Estrutura de Declaração

A construção de uma função segue regras de escopo e interface, garantindo a modularização e reuso de código.

- **Tipo de Retorno:** Deve ser expressamente declarado no início do cabeçalho da função.
- **Identificador e Parâmetros:** Nome exclusivo seguido de variáveis de entrada dentro dos parênteses.
- **Corpo da Função:** Bloco delimitado por chaves { } contendo toda a lógica de execução.
- **Retorno de Valor:** A instrução return envia de volta um dado estritamente compatível.
- **Armazenamento:** Na chamada principal, o resultado é usualmente guardado em variável correlata.

esqueleto.c

```
// 1. Cabeçalho da Função
tipo_retorno nome_funcao(
    tipo_dado variavel_1,
    tipo_dado variavel_2
) {

    // 2. Corpo da Função
    // Instruções e lógica interna

    // 3. Instrução de Retorno
    return valor_processado;

}
```

# O Papel dos Parâmetros em C

---

Os parâmetros são elementos fundamentais para a dinâmica de um programa, atuando como variáveis declaradas diretamente no cabeçalho da função.

A principal finalidade dos parâmetros é estabelecer um canal de comunicação entre as funções auxiliares e a função principal.

Através da passagem de parâmetros, o programa consegue enviar valores específicos para serem processados de forma isolada na função.

## **Definição Crítica: Tipagem Rígida de Parâmetros**

É crucial que as variáveis passadas como argumento no momento da chamada sejam do mesmo tipo de dado definido na listagem de parâmetros da função.

Exemplo: se uma função espera receber um parâmetro inteiro, a variável enviada pelo programa principal também deve ser do tipo inteiro.

# Protótipos de Funções

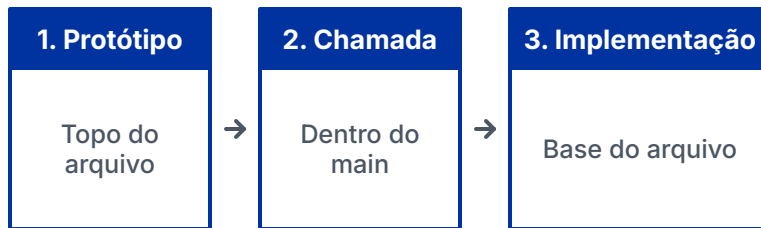
Funções devem ser definidas antes de ser invocada no código.

Contudo, para melhorar a organização e a estrutura do código-fonte, a linguagem C permite a utilização de protótipos de funções.

Um protótipo é uma declaração prévia que informa ao compilador apenas a assinatura da função

Ao declarar os protótipos no início do arquivo, permite ao programador implementar o código **mais tarde**.

Essa prática evita erros de compilação relacionados à ordem de declaração, e das funções disponíveis



```
// 1. Protótipo (no topo do arquivo)
int somar(int a, int b);

// 2. Chamada (dentro da função main)
int main() { somar(5, 3); }

// 3. Implementação (na base do arquivo)
int somar(int a, int b) {
    return a + b;
}
```

# Funções sem Retorno: O Tipo **void**

É possível criar funções que **não retornam nenhum valor**.

Isso ocorre quando o objetivo da função é apenas executar um bloco de comandos (como imprimir um cabeçalho). Nesses casos, utiliza-se a palavra reservada **void** no lugar do tipo de retorno.

Da mesma forma, **void** dentro dos parênteses: a função não necessita os parênteses vazios.

Para o encerramento, utiliza-se a instrução **return;** sem nenhum valor associado.



```
// Protótipo: sem retorno e sem
parâmetros

void imprimirCabeçalho(void);

// Implementação prática da função

void imprimirCabeçalho(void) {
    printf("=====\n");
    printf(" MENU PRINCIPAL \n");
    printf("=====\n");
    return; // Encerramento explícito
}
```

# Vantagens da Modularização em C

---

- **1. Separação de Lógica:** Capacidade de isolar trechos de código que atuam em tarefas específicas, organizando a estrutura lógica geral e tornando o programa principal visivelmente mais limpo.
  - **2. Reuso de Código:** Possibilidade de chamar a mesma função em múltiplos pontos do sistema sempre que necessário, sem qualquer obrigação de reescrever algoritmos complexos.
  - **3. Facilidade de Manutenção:** Essencial para cenários como o cálculo do dígito de um CPF, necessário em 10 locais distintos do sistema. Sem modularização, qualquer alteração na regra exige 10 modificações manuais sujeitas a falhas. Com uma função centralizada, a lógica é alterada em um único ponto e propagada automaticamente por todo o sistema.
-

# Escopo & Variáveis Locais

O conceito de **escopo** determina de forma rigorosa a visibilidade e o ciclo de vida das variáveis dentro de um programa de computador.

Toda função do sistema estabelece um limite de isolamento, permitindo declarar variáveis conhecidas apenas em seu próprio bloco interno de código.

Parâmetros declarados no cabeçalho comportam-se como variáveis locais normais, com a única distinção de inicializarem-se com valores transmitidos pelo programa principal.

Mesmo se existirem identificadores homônimos no programa chamador, o compilador os trata estritamente como entidades de memória distintas.

## Definição Variável Local:

Elemento cujo ciclo de vida e alocação na pilha de memória ocorrem exclusivamente durante a execução de seu escopo correspondente. Ao retornar da função, sua memória é liberada automaticamente e todo o conteúdo é perdido.



# Variáveis Globais

Em contraste direto com as variáveis locais, a linguagem C permite a definição de **variáveis globais**. Declaradas fora do escopo da função, sua principal característica é que seu valor é acessível a partir de qualquer parte do programa abaixo de sua declaração.

Isso viabiliza um **compartilhamento de dados** universal entre múltiplas funções, eliminando a necessidade de passar valores de forma redundante e repetitiva através de parâmetros.

## Atenção: Perda de Controle de Estado



Embora ofereçam conveniência para o acesso amplo, o uso de variáveis globais exige **extrema** cautela. Como qualquer função pode modificar seu valor a qualquer momento, o rastreamento de erros e a manutenção do código tornam-se significativamente complexos em sistemas extensos.

# Global vs. Local

---

```
int year = 2026;

void parse_year() {
    printf("%i\n", year);
}

int main() {
    int year = 2023;
    printf("%i\n", year);
    year = 2021;
    parse_year();
}
```

- identificador
- escopo
- visibilidade

# Passagem de Parâmetros por Valor

## Isolamento e Proteção

Na linguagem C, a **passagem por valor** é o método padrão de comunicação. Quando uma variável é passada como argumento, o sistema envia apenas uma **cópia** do seu valor.

A função trabalha exclusivamente com essa cópia isolada no escopo interno. Qualquer alteração realizada no parâmetro **não modifica a variável original** no programa principal.

Esse comportamento protege as variáveis externas contra efeitos colaterais acidentais, blindando o estado global do fluxo de dados da aplicação.



```
// Demonstração física da cópia
void altera(int x) {
    x = 99; // Altera apenas a cópia
}

int main() {
    int num = 10;
    altera(num); // Envia o valor 10
    // num permanece intacto com 10
    return 0;
}
```

# Passagem de Parâmetros por Referência

## Passagem por Referência

- Passa para a função o **endereço de memória** da variável, viabilizando o acesso de forma direta.
- Permite que a função **modifique diretamente os valores** no escopo original do programa chamador.
- Na linguagem C, essa mecânica de referência exige a utilização de **ponteiros**.

## Passagem por Valor

- **Cópia do valor** da variável.
- **Isolamento total** dos dados: as alterações feitas nos parâmetros não afetam o conteúdo original da variável.
- O escopo da função aloca seu próprio espaço de memória.

**Sintaxe C:** O parâmetro da função deve ser um ponteiro indicado por asterisco (**\*ptr**). Na chamada da função, envia-se o endereço com o sinal (**&var**). Internamente, manipula-se o valor apontado usando novamente o asterisco (**\*ptr**).

# Valor vs. Referência

---

```
int main() {  
    int num = 10;  
    printf("%i\n", num);  
    altera(num);  
    printf("%i\n", num);  
    altera_mesmo(&num);  
    printf("%i\n", num);  
    return 0;  
}
```

```
void altera(int x) {  
    x = 99;  
}  
void altera_mesmo(int *x) {  
    *x = 99;  
}
```

# Passagem por Referência

---

```
int V1=0, V2=0;
```

```
printf("Digite o primeiro valor:");  
scanf("%d", &V1);  
printf("Digite o segundo valor:");  
scanf("%d", &V2);
```

## Vocabulário

- Escopo
- Variável Global
- Variável Local
- Parâmetro
- Protótipo
- Função
- Passagem por referência e por valor
- Ponteiros